Branch · Optimization Lab Assignment

11/06/2026 20:27:34
https://chatgpt.com/c/6aedd57a-93fc-83ec-a845-ab0117feece6

Assignment_8_Optimize_Anything_Vibe_Coding_Challenge.pdf this is my optimization technique lab assignment. fir…

I'll read the assignment using the PDF skill, then explain what you need to build, test, and submit. Your assignment…

PPT3_Quantum_Basics_and_Optimization.pptx PPT1_ML_Optimization.pptx PPT2_Parallel_Optimization.pptx…

I'll use the presentation skill to read all four decks and map Chapter 6 to the assignment, so we can make an…

ok so lets decide that

Let's narrow Track E to three concrete options. I'll compare what each would optimize and how we could measure…

> Which main case study would you like to build? EV charging scheduler — my recommendation

We'll use EV charging scheduling for Track E. I'll define the scope and show a small before-and-after example so yo…

start

I'll build the Track E experiment with a reproducible dataset, baseline, greedy scheduler, integer optimizer, and an…

move to next

I'll move to Track A: continuous optimization. I'll compare the required optimizers on convex and non-convex…

continue

I'll continue with Track B: discrete optimization. I'll compare greedy, dynamic programming, Branch-and-Bound, an…

continue

I'll continue with Track C: quantum optimization. I'll use Track B's exported problems for QAOA, then add VQE, a…

continue

I'll continue with Track D: submodular selection, parallel computing, FedAvg, and multi-task optimization. I'll measu…

Continue

I'll continue with Track F: compiler optimization. I'll build a small compiler, implement four optimization passes, and…

Continue

I'll continue with Track G: xv6 operating-system optimization. I'll first check the RISC-V/QEMU setup, then impleme…

continue

I'll continue with Track H: DBMS optimization, using a real database, measured query performance and correctness…

next

I'll move to Track I: quantum algorithms and QML comparison. I'll check what Track C already covers, then build an…

continue

I'll continue with Track J: software-engineering optimization. I'll use a substantial codebase, profile a concrete…

continue

I'll combine Tracks A–J into one submission package, check them against the assignment and Chapter 6 material, a…

Continue this chat

I'm continuing with the combined report and submission package. The code checks are complete; I'm finishing the…

Yeah, so I will, uh, tell me what should I give? And I will give it to Gemini RCON 4 so that it can give me what. Just…

I'll pull out the exact comparison prompt so you can paste it into Gemini. Thinking ● Reviewed library documentati…

[Image: image(20261105-185811).png] answer "```python from typing import Any def solve_knapsack(weights:…

I'll test Gemini's original code against the same checks, record its roughly 60-second response time as your…

do a full check is everything correct and done properly is this ready to submit like can i just download this add it to…

I'll check the latest report, project ZIP and demo against the assignment requirements, then tell you clearly whethe…

1. can i give you the repo name and you push it . 2. how to do what to do guide . 3. tell me and i will ask him in…

I'll check the GitHub connection, then give you the steps for collecting the AI evidence and the question to ask sir…

Thinking ● Reviewing visible context Reviewing visible context ● Reviewed visible context ● Checking reproductio…

good then complete this i will make a github repo and send you its link in next message

I'll start closing the missing research-method coverage now. I'll check the papers, add experiments we can verify,…

https://github.com/akshattikaria/OT_LAB_ASSIGNMENT_8

I'll check the repository, upload the latest project with the report and demo, then verify the published files. The...

i have given you addces to the repo using the githup plugin

Yes, the GitHub connection confirms write access to your repository. I'm continuing with the upload. The package...

Publish the full assignment publicly

I'll publish the complete assignment to your repository, then verify that the code, results, report and demo are...

Assignment_8_Optimize_Anyth...

this is my optimization technique lab assignment. first tell me what i need to do for this project

10/17/2026, 3:00:43 PM

I'll read the assignment using the PDF skill, then explain what you need to build, test, and submit.

Your assignment is **a collection of 10 compulsory optimization experiments, including one main case study of your choice**. You must use AI-assisted coding, verify the code yourself, and demonstrate through measurements whether each optimization helps. Building an application alone is insufficient. :codex-file-citation{path="sandbox:/workspace/scratch/f3cd84c5991c/upload/Assignment_8_Optimize_Anything_Vibe_Coding_Challenge.pdf" purpose="source"}

The brief gives you **14 days** and expects a public GitHub repository, a technical report, and a short demonstration.

1. Complete all ten tracks

Keep each experiment focused—the assignment explicitly says these do not need to be ten large software projects.

Track	What you need to do	What you must demonstrate
A — Continuous optimization	Compare GD, SGD, mini-batch SGD, Momentum, Nesterov, AdaGrad, RMSProp, Adam, Newton, Gauss–Newton, and the required curvature methods. Include a small ML model with backpropagation.	Convergence, final objective value, runtime, and sensitivity to settings. Include convex and non-convex examples showing local minima or saddle points.
B — Discrete optimization	Choose a selection, scheduling, or allocation problem. Compare greedy, ILP, Branch-and-Bound, and dynamic programming. Also formulate a problem using QUBO/Ising with constraint penalties.	Solution quality and runtime, with an exact solution where feasible. The QUBO problem becomes the classical baseline for Track C.
C — Quantum optimization	Demonstrate VQE, QAOA, parameterized circuits/QNNs, parameter-shift, SPSA, and quantum natural gradient. Solve Track B's QUBO with QAOA.	Comparison with classical results, plus experiments on finite shots, noise, and barren plateaus as circuit size grows. Clearly identify simulator results.
D — Submodular, parallel, and distributed optimization	Demonstrate submodular greedy selection; scheduling and parallel execution; data, model, and pipeline parallelism; FedAvg; and two-task optimization using MGDA and GradNorm.	Speedup, parallel efficiency, overhead, and limits. Compare FedAvg with centralized training, and MGDA/GradNorm with equal task weighting.
E — Your main case study	Choose a real or realistically simulated process, formulate its optimization problem, and compare **at least two methods**.	A complete baseline-to-results investigation, including a method-selection table and discussion of trade-offs. Prefer a domain different from F–J.
F — Compiler optimization	Build a toy compiler or custom compiler passes. Implement **at least two optimizations; four for full credit**. Formulate one compiler decision as an optimization problem solved using a Chapter 6 method.	Instruction count, execution time, code size, and compile time. Optimized and original programs must produce identical outputs.
G — xv6 operating system	Modify a real xv6 kernel feature, such as its scheduler or buffer cache. **One feature across at least three workloads is the**	Before/after benchmarks under QEMU. xv6 must boot and pass `usertests`. Submit the kernel patch or diff and user-level benchmarks.

Track	What you need to do	What you must demonstrate
	minimum; a second feature earns full credit.	
H — DBMS optimization	Optimize a database workload—for example, index selection under a storage budget. Compare **two Chapter 6 methods with the database's default behavior**. A single added index is insufficient.	Query plans, latency, throughput, storage cost, and identical query results on a documented dataset.
I — Quantum algorithms and QML	Run every quantum algorithm taught in class. Write circuits for **at least two algorithms yourself**. Compare a QML model with a classical model on the same data.	Accuracy, training time, parameter count, and circuit evaluations. Compare an algorithm such as Grover with its classical counterpart using query/gate counts and wall-clock time. Include real hardware or a hardware noise model where access allows.
J — Software-engineering optimization	Optimize a realistic codebase, such as test selection, build scheduling, or algorithm selection. Use a named Chapter 6 method against a naive baseline.	Profile the bottleneck, formulate variables/objective/constraints, and measure the result. Use a non-trivial codebase—for example, 1,000+ lines or a public open-source project.

2. Follow the same scientific process for every track

For each experiment, you need to:

1. **Identify the problem:** What wastes time, memory, money, energy, or effort?
2. **Measure a baseline:** How does the original or naive method perform?
3. **Formulate it mathematically:** Define decision variables, objective function, constraints, and assumptions.
4. **Choose suitable methods:** Explain why they fit the problem.
5. **Implement with AI assistance:** Save the prompts and resulting changes.
6. **Verify correctness:** Check outputs, objective values, and constraint satisfaction.
7. **Compare fairly:** Use comparable data, hardware, stopping conditions, and settings.
8. **Explain the result:** Discuss gains, overhead, failures, and limitations.

An optimization is allowed to perform worse. A correct experiment with an honest explanation can still earn full credit. For stochastic methods, repeat runs where feasible and report variability.

3. Cover your exact Chapter 6 syllabus

The tracks are not the entire checklist. You must account for **every method and concept actually taught in Chapter 6**, mapping each to code and experimental evidence.

That includes foundations such as regularization, continuous versus discrete problems, convexity, convergence, and optimization strategy selection. Quantum coverage also depends on what was taught—for example, Deutsch–Jozsa, Bernstein–Vazirani, Grover, QFT, QPE, Shor, annealing, and quantum kernels.

Your Chapter 6 notes/slides are therefore needed before finalizing the implementation checklist. Describing an algorithm without a verifiable demonstration does not satisfy the brief.

4. Document AI use carefully

AI coding is compulsory in every track. You must maintain:

- An **AI tools register** listing tools, model versions, dates, tracks, and uses—including explanation and documentation.
- A `prompts/` **folder with one subfolder per track**, containing prompts, responses/excerpts, revisions, related commits, verification notes, and chat exports or screenshots.
- Git commits distinguishing AI-assisted work from manual corrections.

- **At least three genuine AI failures**, with fixes and lessons.
- A comparison of **at least two AI tools on the same task**, covering correctness, hallucinations, speed, and prompt iterations.

You must also understand the code well enough to **explain and modify it live without AI during the viva**.

5. Submit the complete evidence

Your final submission needs:

- A **public GitHub repository** with setup instructions, dependencies, execution commands, source code, and tests.
- A **Chapter 6 coverage checklist** linking methods to experiments, code, and results.
- Reproducible **results, tables, and plots** with units and settings.
- The complete AI disclosure and prompt records.
- A **technical report with a section for each track A–J**, including formulations, baselines, correctness checks, results, limitations, and answers to the ten final questions in the brief.
- A **short demonstration** explaining the problem, methods, evidence, and lessons.

The practical first step is to turn your Chapter 6 material into a coverage checklist, choose Track E's process, and define one small, measurable experiment for each remaining track. This keeps the work manageable while satisfying the full assignment.

10/17/2026, 3:00:55 PM · gpt-6.1-sol-wm

PPT3_Quantum_Basics_and_O...

PPT1_ML_Optimization.pptx

PPT2_Parallel_Optimization.pptx

PPT4_Strategy_Selection.pptx

this is the complete chapter 6\

10/31/2026, 1:26:20 AM

I'll use the presentation skill to read all four decks and map Chapter 6 to the assignment, so we can make an accurate coverage checklist.

Thinking

- **Worked**
- **Worked**
- **Explored files and ran 2 commands**

✓ Worked for 31s

I've reviewed all four decks—146 slides in total. This gives us the Chapter 6 material needed to plan the project accurately.

One important finding: **the assignment asks for more than the slides teach**, so we need to cover both the slide content and the additional requirements explicitly stated in the assignment.

The chapter breaks down as follows:

Chapter section	Topics covered	Where they fit in your project
ML optimization	Loss functions, convex/non-convex landscapes, stationary points, GD, SGD, mini-batch SGD, backpropagation, Momentum, Nesterov, AdaGrad, RMSProp, Adam, Newton, convergence, forward/backward feature selection, ILP, L0/L1/L2 regularization, submodular greedy selection.	Primarily **Tracks A, B, and D**, with methods reused in your main case study.
Parallel optimization	Static, cyclic, dynamic, guided scheduling and work stealing; makespan; cache blocking and locality; false sharing; synchronization; MPI collectives; batching and overlapping communication; local, synchronous, and asynchronous SGD; Amdahl's and Gustafson's laws.	Primarily **Track D**, with useful techniques for F–J.
Quantum basics and optimization	Qubits, gates, measurement, interference, entanglement, Deutsch's algorithm, Grover, parameterized circuits, VQE, QAOA, QNN/variational classifiers, parameter-shift, QUBO/Ising, shots, noise, and barren plateaus.	**Tracks B, C, and I**.

Chapter section	Topics covered	Where they fit in your project
Strategy selection	Choosing methods by problem type and scale; L-BFGS; data/model/tensor/pipeline parallelism; all-reduce; communication compression; hybrid layouts; grid, random, Bayesian, and successive-halving hyperparameter search.	Your **method-selection justification**, Track D, and suitable experiments in E or J.

These mappings come from the ML deck :codex-file-citation{path="sandbox:/workspace/scratch/f3cd84c5991c/upload/PPT1_ML_Optimization.pptx" purpose="source"}, parallel deck :codex-file-citation{path="sandbox:/workspace/scratch/f3cd84c5991c/upload/PPT2_Parallel_Optimization.pptx" purpose="source"}, quantum deck :codex-file-citation{path="sandbox:/workspace/scratch/f3cd84c5991c/upload/PPT3_Quantum_Basics_and_Optimization.pptx" purpose="source"}, and strategy-selection deck :codex-file-citation{path="sandbox:/workspace/scratch/f3cd84c5991c/upload/PPT4_Strategy_Selection.pptx" purpose="source"}.

Three points change how we should plan the work:

1. **Quantum remains compulsory for this assignment.**
   The slides label quantum as “optional enrichment,” but the assignment explicitly makes all ten tracks compulsory, including C and I. We should follow the assignment unless your professor gives a specific exemption.

2. **Some required methods need supplementary learning.**
   Gauss–Newton, SPSA, quantum natural gradient, FedAvg, MGDA, and GradNorm are not explained in these decks, but the assignment explicitly requires them. The brief also requires curvature demonstrations beyond the slides’ basic Newton treatment. These must remain on our checklist.

3. **We should distinguish actual taught algorithms from examples in the brief.**
   The quantum deck teaches **Deutsch’s algorithm and Grover**. Deutsch–Jozsa, Bernstein–Vazirani, QFT, QPE, Shor, and quantum annealing are not taught in these four decks. The assignment introduces that list as examples of additional algorithms taught in class, so we should not automatically treat every example as an additional compulsory implementation. However, explicitly named track requirements still apply. :codex-file-citation{path="sandbox:/workspace/scratch/f3cd84c5991c/upload/Assignment_8_Optimize_Anything_Vibe_Coding_Challenge.p purpose="source"}

For planning, we should maintain a checklist with **topic, slide reference, assignment requirement, experiment, code location, and result**. Research spotlights such as AdaFisher and GRNG should have separate entries so they are accounted for without confusing a brief research mention with a fully taught implementation.

You now have the syllabus inputs covered. The next decision is the main process for Track E; the other nine tracks can use small, focused experiments, sharing datasets and benchmarks where appropriate.

10/31/2026, 1:26:36 AM · gpt-6.1-sol-wm

ok so lets decide that

10/31/2026, 3:54:55 AM

Let’s narrow Track E to three concrete options. I’ll compare what each would optimize and how we could measure improvement, then use the question-picker skill to help us choose.

Thinking

- **Worked**
- **Searched the web**

✓ Worked for 2m 23s

My recommendation is a Smart EV Charging Scheduler for Track E. It gives us a clear optimization problem, measurable results, and an easy visual demonstration.

These are the three options I would shortlist:

Option	What we optimize	What you could show your professor
EV charging scheduler	When vehicles charge, to reduce electricity cost while meeting departure deadlines and power limits.	Before/after charging schedules, electricity costs, and power-demand graphs.
College exam timetable	Exam and room assignments, avoiding clashes and reducing back-to-back exams.	Two timetables with a comparison of student inconvenience and room usage.
Delivery-route planner	Which vehicle visits each destination and in what order, subject to carrying capacity.	Before/after routes and total distance travelled.

All three have useful public reference material: Caltech provides EV charging-session data, UniTime publishes examination datasets, and Google provides a worked capacity-constrained routing example. (ev.caltech.edu, unitime.org, developers.google.com)

For the **EV option**, the idea is:

> *Several vehicles are plugged into a charging station. Each needs a certain amount of energy before its departure time. Electricity prices vary during the day, and the station has limited power. Decide when each vehicle should charge.*

We would build a **small software simulation**, initially using roughly 10-30 vehicles and clearly documented assumptions.

Component	Proposed approach
Baseline	Charge vehicles as early as possible, respecting the station’s power limit.
Method 1: Greedy	Schedule vehicles by urgency, selecting inexpensive available charging periods.
Method 2: Integer optimization	Choose charging periods jointly for all vehicles using an ILP/MILP formulation.
Correctness checks	Vehicles receive their required energy before departure; charging and station power limits hold.
Comparison	Electricity cost, energy delivered, missed deadlines, peak power, and computation time.

Why I favour this: the objective has a concrete meaning-electricity cost-and the constraints are easy to explain during a viva. We can investigate *when* optimization helps by changing prices, demand, and available power. Lower cost would only count as an improvement when we preserve the required charging service.

This would be your **Track E case study**; the other tracks would retain their own focused experiments.

Sources

ACN-Data -- A Public EV Charging Dataset ev.caltech.edu

UniTime | Examination Timetabling Benchmark Benchmark Datasets unitime.org

Capacity Constraints | OR-Tools | Google for Developers developers.google.com

10/31/2026, 3:55:01 AM · gpt-6-astra-wm

> Which main case study would you like to build?
EV charging scheduler — my recommendation

10/31/2026, 3:58:34 AM

We'll use EV charging scheduling for Track E. I'll define the scope and show a small before-and-after example so you can see exactly what the optimizer will do.

The scope will be fixed-rate charging in time slots, with each vehicle's energy need and departure deadline known upfront. I'm using the visualization skill to show how shifting flexible charging to cheaper hours can reduce cost while still delivering the same energy on time.

Thinking

- **Worked**
- **Checking SciPy and NumPy availability**
- **Ran 4 commands and edited 1 file**

✓ Worked for 4m 40s

Track E is decided: "Smart EV Charging Scheduling under Power and Departure Constraints."

We'll optimize **when each vehicle charges**, aiming to reduce electricity cost while supplying its required energy before departure.

The first version will use these boundaries:

- **Inputs:** arrival times, departure deadlines, energy requirements, electricity prices, and charger/station power limits.
- **Decision:** whether each vehicle charges during each time slot.
- **Assumption:** vehicles remain connected to assigned plugs, and all session requirements are known beforehand.
- **Methods:** an earliest-charging baseline, a deadline-aware greedy scheduler, and an integer optimization model.
- **Evidence:** cost, energy delivered, missed deadlines, power-limit violations, and computation time.

Here is a small **synthetic example with fictional prices and hourly slots**. Change the price setting to see when shifting charging helps.

visualize

For the actual experiment, we'll start with **10-30 vehicles and 15-minute slots**, then vary demand, available power, prices, and deadline flexibility. Flat prices and tight deadlines will help us identify situations where optimization provides little benefit.

The first implementation milestone will be a **reproducible dataset generator, baseline scheduler, and independent correctness checker**. That gives us a reliable foundation before adding the optimization methods.

10/31/2026, 3:58:40 AM · gpt-6-astra-wm

start

10/31/2026, 2:17:51 PM

I'll build the Track E experiment with a reproducible dataset, baseline, greedy scheduler, integer optimizer, and an independent correctness checker. Then I'll run comparisons and package the code with its results.

I'll start with clearly labelled synthetic data. Every schedule will be checked for energy delivery, deadlines, and power limits; a cheaper schedule that leaves a vehicle undercharged won't count as an improvement. I'm also using the Library skill to preserve the runnable project and its AI prompt record.

The core schedulers pass 29 tests, including checks against exhaustive search on 80 small random cases. A useful test case shows greedy choosing a cheap slot that another vehicle needs to meet its deadline. The checker correctly rejects that schedule, so we can report this limitation clearly.

The benchmark has finished: 75 main scenarios plus four diagnostic cases. The integer solver produced valid schedules for all 75, proving optimality for 70; five reached the three-second limit with valid schedules still returned. Flexible-window cases averaged 49.4% lower cost than the baseline, while flat-price cases showed no savings. These are synthetic-data results.

The code, report, plots, and raw results are ready. I've rechecked every saved schedule against its input and verified the summary tables. I'm now checking the run commands and packaging this Track E milestone, with the remaining assignment requirements listed clearly.

Thinking

- **Worked**
- **Ran 2 commands and searched the web**
- **Ran 2 commands and edited 10 files**
- **Ran 3 commands and edited 3 files**
- **Explored files, edited 7 files, and ran 2 commands**
- **Edited 1 file, ran 6 commands, and used Library**

✓ Worked for 18m 50s

The Track E prototype is built and tested.

- Includes baseline, greedy, and integer optimization methods.
- **29 tests passed**, including checks against exhaustive search.
- Tested **75 main scenarios**: all integer-solver schedules were valid; 70 were proven optimal and five reached the time limit.
- Flexible-window cases averaged **49.4% lower cost**; flat-price cases showed no savings. These use synthetic data.

Track_E_EV_Charging_Project.zip

The package contains code, datasets, raw results, charts, mathematical formulation, AI records, and a viva guide.

To see the results: extract the ZIP and open `ev_charging_track_e/results/report.html`. The README contains execution instructions.

The other nine assignment tracks remain.

10/31/2026, 2:18:17 PM · gpt-6-astra-wm

move to next

10/31/2026, 11:38:06 PM

I'll move to Track A: continuous optimization. I'll compare the required optimizers on convex and non-convex problems, then add backpropagation, Newton/Gauss–Newton, curvature checks, and tuning experiments. Track E will remain intact.

The Track A implementations pass their first 29 tests, including numerical checks of backpropagation and the second-order derivatives. I also found an arithmetic error in the supplied SGD example: its stated data and loss give a gradient of −3 and a first step of 0.3. I'll document that correction.

The main benchmark is complete: 270 training runs across five data seeds, with learning rates selected on a separate tuning dataset. Newton and Gauss–Newton reach the ridge optimum in one update, as expected for that quadratic problem. The neural-network, noisy-objective, saddle-point, and curvature experiments have also finished; I'm checking the results and plots.

I caught a precision issue in the convergence controls: subtracting nearly equal numbers made a tiny positive loss appear to be zero. I'm correcting the formula and adding a regression test. I'm also adjusting the log-scale plots so their uncertainty bands are easier to read.

All 30 tests now pass. I've also checked the saved results against their datasets and confirmed that the earlier Track E archive is unchanged. I'm packaging Track A with its report, plots, runnable code, and the documented numerical fix.

Thinking

- **Worked**
- **Ran 4 commands, searched the web, and edited 9 files**
- **Edited 3 files and ran 3 commands**
- **Preparing Baseline Snapshot and Commit**
- **Edited 11 files and ran 7 commands**
- **Ran 5 commands and used Library**

✓ Worked for 17m 25s

Track A's core experiments are built and tested.

Track_A_Continuous_Optimization_Project.zip

Includes:

- All eight gradient methods, plus Newton and Gauss–Newton.
- Backpropagation, curvature checks, convergence, tuning, and noise experiments.
- **270 main training runs and 30 passing tests.**
- Code, raw results, plots, report, and viva guide.
- One genuine numerical bug documented and fixed.

Extract the ZIP and open `continuous_track_a/results/report.html`. The coverage checklist flags research-paper reproductions that remain pending.

Track E is preserved. **Next is Track B: discrete optimization.**

10/31/2026, 11:38:08 PM · gpt-6.1-sol-wm

continue

11/3/2026, 11:15:28 AM

I'll continue with Track B: discrete optimization. I'll compare greedy, dynamic programming, Branch-and-Bound, and ILP on a selection problem, then build and verify a QUBO/Ising version for use in Track C.

I'll use budgeted item selection for the main comparison, with a separate small feature-selection experiment. The QUBO section will include a penalty-strength check, so we can show when an energy minimum does—and does not—represent a valid constrained solution.

The exact methods agree with exhaustive search on 100 small random problems. I also found a precision issue with very large QUBO coefficients: the expanded energy can lose a small objective difference. I'm adding a range check and a regression test so it rejects those inputs explicitly.

The benchmark is complete: 80 generated instances and five diagnostic cases, with three repeats per method. DP and ILP proved the optimum on all 80 main instances; Branch-and-Bound proved 73 within its limits and returned feasible candidates for the rest. Both quantum-ready models also passed the QUBO/Ising energy checks for every bitstring.

Track B now passes 30 tests, including feature-selection data-isolation checks and the QUBO precision regression. I'm verifying the saved results and packaging the two classical baselines with explicit bit order, spin signs, and offsets for Track C.

Thinking

- **Worked**
- **Ran 3 commands and searched the web**
- **Edited 8 files and ran 3 commands**
- **Ran 5 commands and edited 5 files**
- **Edited 9 files and ran 3 commands**
- **Ran 6 commands, edited 2 files, and used Library**

✓ Worked for 16m 27s

Track B is built and tested.

Track_B_Discrete_Optimization_Project.zip

Includes:

- Greedy, dynamic programming, Branch-and-Bound, and ILP comparisons.
- **1,275 recorded runs and 30 passing tests.**
- Forward/backward feature selection.
- Verified QUBO/Ising models and classical baselines for Track C.
- Code, raw results, plots, report, and viva guide.

DP and ILP proved all **80 main optima**; Branch-and-Bound proved **73 within its limits**. One additional numerical defect was fixed and documented.

Extract and open `discrete_track_b/results/report.html`.

Tracks A and E remain preserved. **Next is Track C: quantum optimization.**

11/3/2026, 11:16:03 AM · gpt-6.1-sol-wm

continue

11/3/2026, 11:33:32 AM

I'll continue with Track C: quantum optimization. I'll use Track B's exported problems for QAOA, then add VQE, a trainable quantum circuit, gradient methods, and experiments on measurement shots, noise, and barren plateaus. These will be simulator experiments, clearly labelled as such.

No quantum SDK is installed here. I'll build a small NumPy simulator and check its gates and circuits against independent dense-matrix calculations. I'll also count circuit evaluations and shots separately, so simulator runtime cannot be mistaken for quantum hardware performance.

The simulator is ready, and the first checks match the expected circuit behavior. I'm now testing the gradients, noise channels, and Track B energy mappings before running the full experiments and building the report.

All 32 correctness tests passed. The experiments also show a useful limitation: deeper QAOA improved Max-Cut, but performed poorly on the constrained knapsack under this search setup. I'll retain every seed in the report and show the extra metric cost paid by quantum natural gradient.

The full experiments and charts are complete. The report separates expected energy, sampled solution quality, and constraint feasibility. I'm checking the retained results and packaging the code, tests, report, and viva notes.

The result audit passed: 496 retained VQE iterates, 4,898 QAOA evaluations, and all 1,600 sampled distributions checked out. The package is nearly ready. It includes reproducible commands and viva notes, with the simulator limits and poor knapsack results clearly explained.

Thinking

- **Worked**
- **Ran 2 commands and searched the web**
- **Edited 9 files, ran 5 commands, and explored files**
- **Ran 5 commands, edited 2 files, and explored files**
- **Edited 8 files, ran 3 commands, and explored files**
- **Ran 8 commands and edited 4 files**
- **Edited 1 file, ran 3 commands, and used Library**

✓ Worked for 20m 8s

Track C is ready: VQE, QAOA, QNN, gradient methods, and shot/noise/barren-plateau experiments—all using a CPU simulator.

32 tests and the results audit passed. Includes code, charts, report, and viva notes.

Track_C_Quantum_Optimization_Project.zip

After extracting, open `quantum_track_c/results/report.html`.

Next is **Track D: submodular, parallel and distributed optimization**.

11/3/2026, 11:33:38 AM · gpt-6.1-sol-wm

continue

11/3/2026, 5:39:57 PM

I'll continue with Track D: submodular selection, parallel computing, FedAvg, and multi-task optimization. I'll measure performance against the workspace's actual CPU allocation and include the report, code, results, and viva notes.

This workspace has an eight-CPU quota, so I can test 1, 2, 4, and 8 worker processes. I'll also compare FedAvg at equal training work and equal communication rounds, since those answer different questions.

The selection and learning methods are implemented; the first check confirms that one-step FedAvg matches centralized training even with unequal client sizes. I also caught a timing issue: the "steady" timer included workers finishing startup. I'll correct that before benchmarking and retain the finding in the AI error log.

The first test run found an environment limitation: socket-based multiprocessing managers are blocked here. I'll use shared-memory arrays and locks for the stealing queues, then rerun the tests before collecting parallel timings. The learning experiments have already finished.

All 29 tests pass, and the process benchmarks are running. In the learning results, longer local FedAvg runs reduced communication but increased error on skewed clients. MGDA and GradNorm also produced different task trade-offs; I'll report both task errors so the comparison stays clear.

The benchmarks and audit are complete. On the irregular workload, the eight-worker task pool achieved 5.72× steady speedup, though startup erased the end-to-end gain for these short jobs. Data parallel inference reached about 3× at the larger batch size, and the best measured two-stage pipeline reached 1.48×. I'm finishing the report and package checks.

The fresh CLI run passed too. The package now includes seven charts, all raw results, and the corrected timing evidence. The audit passed 25,253 numerical checks. I'm packaging the final files; next is Track F, since the EV case study in Track E was completed first.

Thinking

- **Worked**
- **Searched the web, ran 3 commands, and explored files**
- **Ran 4 commands, searched the web, and edited 10 files**
- **Ran 8 commands and edited 5 files**
- **Ran 7 commands and edited 3 files**
- **Edited 2 files and ran 3 commands**
- **Ran 5 commands and edited 10 files**
- **Ran 3 commands and used Library**

✓ Worked for 22m 9s

Track D is ready: submodular selection, CPU parallelism, FedAvg, MGDA and GradNorm.

29 tests passed, along with 25,253 audit checks. Includes code, raw results, charts, report and viva notes.

Track_D_Parallel_Distributed_Optimization_Project.zip

After extracting, open `distributed_track_d/results/report.html`.

Next is **Track F: compiler optimization**—Track E is already complete.

11/3/2026, 5:40:28 PM · gpt-6.1-sol-wm

Continue

11/4/2026, 10:56:19 AM

I'll continue with Track F: compiler optimization. I'll build a small compiler, implement four optimization passes, and compare the generated programs for correctness, runtime, and code size.

I'll use a small unsigned-integer language with four passes: constant folding/propagation, common-subexpression elimination, dead-code elimination, and strength reduction. Dynamic programming will choose the pass order, with greedy selection and exhaustive search as comparisons. I'll also compare native code at `-O0` and `-O2` to show how the host compiler affects the results.

The compiler passes all 29 tests, including overflow and shift edge cases, comparisons between the interpreters and native code, and all 24 pass orders. DP also matches exhaustive search on the test programs. Next I'll benchmark repeated native execution and compile time.

The measurements are complete. The passes reduced IR size and improved native runtime at `-O0`. At `-O2`, the extra benefit was small or negative, so I'll keep both results visible. DP found a lower weighted IR cost than greedy ordering on 10 of the 20 programs.

The audit passed 24,400 comparisons, including fresh native builds and all saved pass-order certificates. The public compile and benchmark commands also passed a fresh smoke run. I'm finishing the explanations and viva notes, then packaging Track F.

Track F is ready for packaging. It includes the compiler, generated IR/C and disassembly, raw results, six charts, and viva notes. The earlier five track archives are unchanged; the report clearly separates the DP cost-model optimum from measured native performance.

Thinking

- **Finding Project Guidance And Coverage Files**
- **Inspected project guidance, coverage files, Python environment, and system resources**
- **Searching assignment8 compiler sections**
- **Designing pass-order DP**
  Designing pass-order DP
- **Defining straight-line IR**
  Defining straight-line IR
- **Designed pass-order dynamic programming and defined straight-line IR**
- **Defining arithmetic semantics**
  Defining arithmetic semantics
- **Defining VM benchmarks**
  Defining VM benchmarks
- **Defined arithmetic semantics and VM benchmarks**
- **Defining lexer grammar**
  Defining lexer grammar
- **Defining AST model**
  Defining AST model
- **Defined lexer grammar and AST model**
- **Defining SSA IR passes**
  Defining SSA IR passes
- **Refining CF alias handling**

Refining CF alias handling

- **Defined SSA IR passes and refined control-flow alias handling**
- **Preparing CSE pass**
  Preparing CSE pass
- **Preparing constant-map input**
  Preparing constant-map input
- **Prepared CSE pass and constant-map input**
- **Defining canonical IR states**
  Defining canonical IR states
- **Defining DP state transitions**
  Defining DP state transitions
- **Defined canonical IR states and DP state transitions**
- **Implement memoized order DP**
  Implement memoized order DP
- **Generate the timing harness**
  Generate the timing harness
- **Implemented memoized order DP and generated timing harness**
- **Checking noinline support**
  Checking noinline support
- **Defining disassembly counts**
  Defining disassembly counts
- **Checked noinline support and defined disassembly counts**
- **Planning benchmark parameters**
  Planning benchmark parameters
- **Checking native checksums**
  Checking native checksums
- **Planned benchmark parameters and checked native checksums**
- **Preparing benchmark input generation**
  Preparing benchmark input generation
- **Defining serialized VM instructions**
  Defining serialized VM instructions
- **Prepared benchmark input generation and defined serialized VM instructions**
- **Planning VM implementation**
  Planning VM implementation
- **Verifying baseline archives**
  Verifying baseline archives
- **Planned VM implementation and verified baseline archives**
- **Preparing per-variant measurements**
  Preparing per-variant measurements
- **Preparing per-program artifacts**
  Preparing per-program artifacts
- **Prepared per-variant measurements and program artifacts**
- **Searched assignment8 compiler sections**

- **Creating toy compiler project directories**
- **Created toy compiler project directories**
- **Applying a patch to files**
- **Fixing duplicate greedy evaluations**
  Fixing duplicate greedy evaluations
- **Instrumenting greedy call counts**
  Instrumenting greedy call counts
- **Fixed duplicate greedy evaluations and instrumented call counts**
- **Fixing redundant compile work**
  Fixing redundant compile work
- **Fixed redundant compile work**
- **Committing frontend, SSA, and DP implementation**
- **Choosing argument-load cost**
  Choosing argument-load cost
- **Updating argument cost**
  Updating argument cost
- **Updated argument-load cost**
- **Writing core parser tests**
  Writing core parser tests
- **Adding native arity checks**
  Adding native arity checks
- **Added parser tests and native arity checks**
- **Applying a code patch**
- **Checking full test results**
  Checking full test results
- **Checking native compiler version**
  Checking native compiler version
- **Checked full test results and native compiler version**
- **Defining compile_variant metadata**
  Defining compile_variant metadata
- **Exporting uint64 test vectors**
  Exporting uint64 test vectors
- **Defined compile_variant metadata and exported uint64 test vectors**
- **Preparing native reference checks**
  Preparing native reference checks
- **Preparing native benchmark runs**
  Preparing native benchmark runs
- **Prepared native reference checks and benchmark runs**
- **Checking test output and compiler version**
- **Applied patches, compared compiler codegen, committed fixes, and ran tests**
- **Applying a patch to files**
- **Running native benchmark loop**

Running native benchmark loop

- **Checking native inspection**
  Checking native inspection
- **Ran native benchmark loop and inspection**

🌐 **Running ToyCC protocol benchmark**

- **Implementing audit checks**
  Implementing audit checks
- **Implemented audit checks**
- **Parsing assembly function headers**
  Parsing assembly function headers
- **Parsed assembly function headers**

🌐 **Applying a code patch**

- **Planning report figures**
  Planning report figures
- **Plotting pass-search heatmap**
  Plotting pass-search heatmap
- **Planned report figures and plotted pass-search heatmap**
- **Preparing ablation plots**
  Preparing ablation plots
- **Planning six report figures**
  Planning six report figures
- **Prepared ablation plots and planned six report figures**
- **Preparing examples/demo.toy**
  Preparing examples/demo.toy
- **Auditing C rebuild results**
  Auditing C rebuild results
- **Prepared demo examples and audited C rebuild results**
- **Applied patches, ran tests and benchmarks, reviewed results, and committed changes**

🌐 **Auditing ToyCC compilation results**

- **Preparing IR ratio heatmap**
  Preparing IR ratio heatmap
- **Prepared IR ratio heatmap**

🌐 **Applying a patch to files**

- **Checking plot images**
  Checking plot images
- **Preparing Track F workspace**
  Preparing Track F workspace
- **Checked plot images and prepared Track F workspace**
- **Verifying corrected search accounting**
  Verifying corrected search accounting
- **Validating test vectors**
  Validating test vectors

- **Verified corrected search accounting and validated test vectors**
- **Checking generated C output**
  Checking generated C output
- **Checked generated C output**
- **Running toycc benchmark smoke test**
- **Revising the convergence claim**
  Revising the convergence claim
- **Adding measured O2 results**
  Adding measured O2 results
- **Revised the convergence claim and added measured O2 results**
- **Verifying DP state counts**
  Verifying DP state counts
- **Checking native artifact**
  Checking native artifact
- **Verified DP state counts and checked native artifact**
- **Updating Optimization Report Text**
- **Refining README wording**
  Refining README wording
- **Checking CSE semantics**
  Checking CSE semantics
- **Refined README wording and checked CSE semantics**
- **Checking compile-time accounting**
  Checking compile-time accounting
- **Checking preservation hashes**
  Checking preservation hashes
- **Checked compile-time accounting and preservation hashes**
- **Preparing documentation validation**
  Preparing documentation validation
- **Preparing validation patch**
  Preparing validation patch
- **Prepared documentation validation patch**
- **Auditing native benchmark counts**
  Auditing native benchmark counts
- **Audited native benchmark counts**
- **Audited ToyCC results, applied patches, ran benchmarks, and updated reports**
- **Applying a code patch**
- **Rescaling heatmap colors**
  Rescaling heatmap colors
- **Rescaled heatmap colors**
- **Updating Report Visualization and Index**
- **Awaiting current work**
  Awaiting current work

- **Awaited current work updates**
- **Applied patches, updated reports, cleaned helpers, validated hashes, and committed results**
- **Committing and bundling project provenance evidence**
- **Packaged and validated the compiler project archive with provenance evidence**
- **Created compiler optimization project archive**
- **Applying Library File Metadata**
- **Applied library file metadata**

✓ Worked for 24m 7s

Track F is ready: a toy compiler with four optimization passes, DP pass ordering, and native benchmarks.

29 tests passed, plus 24,400 audit comparisons. Includes code, raw results, six charts, report and viva notes.

`Track_F_Compiler_Optimization_Project.zip`

After extracting, open `compiler_track_f/results/report.html`.

Next is **Track G: xv6 operating-system optimization**.

11/4/2026, 10:56:26 AM · gpt-6.1-sol-wm

Continue

11/4/2026, 1:48:35 PM

I'll continue with Track G: xv6 operating-system optimization. I'll first check the RISC-V/QEMU setup, then implement a kernel change, benchmark it across three workloads, and run xv6's correctness tests.

QEMU and the RISC-V compiler aren't installed here yet. I'm checking whether they can be added so this track can be tested on xv6 itself. If that's blocked, I'll keep the missing kernel validation explicit.

The local toolchain is working: QEMU 8.2.2 and RISC-V GCC are ready, and I've verified all 82 files in the pinned MIT xv6 source snapshot. I'll test two changes: per-CPU page allocation to reduce lock contention, and hashed buffer lookup to reduce cache-search work. Each can be disabled for comparison.

The pristine xv6 build boots and passes the full `usertests` suite. The modified build caught an API mistake: this xv6 version requires a process-size argument in `copyout`. I'm correcting the syscall, retaining the failure log, and then testing all four kernel configurations.

The instrumented baseline has passed all 74 upstream tests and the custom allocator/cache checks. The per-CPU allocator configuration is now in the full test suite. I've also defined six benchmark workloads: three for allocation/fork/pipe activity and three for buffer-cache access, with data checks in each.

The baseline and both individual kernel changes have passed all 74 xv6 tests. I'm checking the combined version next, then I'll run the measurements and finish the report and submission archive.

All four configurations now pass the full xv6 suite and the custom allocator/cache checks. The benchmark is running across six workloads with three repetitions each. I'll report lookup work, lock contention and elapsed time separately so the conclusions reflect what each change actually improves.

The measurements so far show lower allocator contention with per-CPU lists and fewer buffer comparisons with hashing. Elapsed times vary by workload. I'm retaining all repetitions and checking the saved results against the raw QEMU output.

The 72-run benchmark has passed its audit. Hashing reduced lookup comparisons by about 47–86%. Per-CPU allocation cut lock contention, but its page workload ran about 12× slower in QEMU. I've made that regression prominent in the report; its cause is still unresolved.

The report, patch, raw logs and viva notes are ready, and a separate small run confirms the command-line workflow works. I'm packaging the submission now. The earlier track archives are unchanged; Track H, DBMS optimization, is next.

Thinking

- **Worked**
- **Searched the web and ran 5 commands**
- **Ran 8 commands, used GitHub, and edited 83 files**
- **Ran 6 commands, explored files, and edited 18 files**
- **Edited 7 files, ran 7 commands, and explored files**
- **Ran 7 commands, searched the web, and edited 5 files**
- **Ran 3 commands, explored files, and edited 1 file**
- **Explored files, ran 4 commands, and edited 9 files**
- **Ran 10 commands, explored files, and edited 2 files**
- **Ran 11 commands and edited 4 files**
- **Ran 10 commands, edited 1 file, and used Library**

✓ Worked for 70m 35s

Track G is ready: **two xv6 kernel features, six workloads and 72 benchmark runs**. All four configurations passed all 74 upstream tests.

`Track_G_Xv6_OS_Optimization_Project.zip`

Includes source, patch, raw logs, report and viva notes. Open `results/report.html` after extraction.

Hashing reduced lookup comparisons by **47-86%**. Per-CPU allocation reduced contention but made the page workload about **12× slower in QEMU**; the report explains this result.

Next: **Track H - DBMS optimization.**

11/4/2026, 1:49:11 PM · gpt-6.1-sol-wm

continue

11/4/2026, 4:19:18 PM

I'll continue with Track H: DBMS optimization, using a real database, measured query performance and correctness checks.

I'm using a synthetic EV-booking dataset in SQLite and treating index selection as a storage-constrained optimization problem. Greedy selection and an ILP solver will choose among six candidate indexes. I'll compare their query plans, latency, throughput and storage against SQLite's default planner, while checking that every query returns identical results.

A correctness test caught a result mismatch when switching a status parameter while a partial index was present. I'm isolating the cause before running the main experiment. This is why the test checks returned rows across every candidate index combination, not just query speed.

The main run is complete on 200,000 bookings. Greedy and ILP chose the same indexes at all three storage budgets, and exhaustive enumeration confirms those choices are optimal for the stated cost model. The selected indexes improved read throughput substantially, while insert throughput fell. The report will show both effects.

The final audit found three trial blocks missing from the saved checkpoint files, so I won't use that run for the report. I'm rerunning the full protocol with a separate record for every trial and a completeness check after saving.

The complete rerun passed the audit, including fresh SQL checks against the independent reference results. At the 60% budget, the ILP-selected indexes reached about 33× baseline read throughput using 12.8 MiB of index storage. Insert throughput fell to about 18% of baseline. I'm finishing the report and packaging the evidence.

The report, query plans, raw measurements, database and viva notes are ready. All 17 tests passed, and the final audit passed 1,991 checks. I'm packaging Track H now; the earlier track archives remain unchanged.

Thinking

- **Worked**
- **Ran 4 commands and searched the web**
- **Edited 8 files, ran 2 commands, and explored files**
- **Ran 13 commands, edited 4 files, and searched the web**
- **Edited 1 file, ran 3 commands, and explored files**
- **Ran 13 commands, edited 11 files, and explored files**
- **Edited 7 files and ran 9 commands**
- **Ran 9 commands, edited 1 file, and used Library**

✓ Worked for 26m 4s

Track H is ready: **greedy versus ILP index selection in SQLite**, using 200,000 bookings. All **17 tests and 1,991 audit checks passed**.

`Track_H_DBMS_Optimization_Project.zip`

Includes code, database, query plans, raw measurements, report and viva notes. Open `results/report.html` after extraction.

At the 60% storage budget, measured read throughput reached **33× baseline**, while insert throughput fell to **18% of baseline**.

Next: **Track I - quantum algorithms and QML comparison.**

11/4/2026, 4:19:57 PM · gpt-6.1-sol-wm

next

11/4/2026, 5:11:45 PM

I'll move to Track I: quantum algorithms and QML comparison. I'll check what Track C already covers, then build and test the remaining experiments.

Track C already covers VQE, QAOA, parameter-shift, SPSA, quantum natural gradient and gradient-concentration experiments. Track I will add the missing algorithm circuits, a quantum-kernel versus classical-kernel comparison, and noise simulation based on a saved device calibration. I'll distinguish oracle-query counts from simulator wall-clock timings.

All 22 circuit and mathematics tests passed, and the small end-to-end run is complete. It caught a metadata-writing error, which I fixed before the main run. The main experiments are now running with the historical Manila calibration dated 27 May 2024; no physical quantum processor is being used.

The experiments are complete. The quantum kernel scored below the classical RBF kernel on both datasets and took longer to simulate. Grover used 12 search-oracle calls for 256 candidates, but its simulator run was much slower than linear search. The audit also reproduced all 24 seeded Aer noise/control runs.

The report and viva notes are ready. All 23 tests and 1,803 audit checks passed, including the noise replays. I'm packaging the circuits, raw results, data splits, model parameters and verification records. The earlier track archives are unchanged.

Thinking

- **Worked**
- **Explored files, ran 5 commands, and searched the web**
- **Ran 15 commands, searched the web, edited 12 files, and explored files**
- **Edited 3 files, explored files, and ran 5 commands**
- **Edited 13 files, ran 13 commands, explored files, and searched the web**
- **Edited 5 files, ran 14 commands, and used Library**

✓ Worked for 42m 25s

Track I is ready: quantum circuits, historical calibrated-noise simulation and QML/classical comparisons. **23 tests and 1,803 audit checks passed.**

`Track_I_Quantum_Algorithms_QML_Project.zip`

Includes code, raw results, circuit diagrams, report and viva notes. Open `results/report.html` after extraction.

The quantum kernel underperformed RBF on both datasets. Grover's oracle-query savings are clearly separated from simulator timings.

Next: **Track J - software-engineering optimization.**

11/4/2026, 5:11:59 PM · gpt-6.1-sol-wm

continue

11/4/2026, 5:56:55 PM

I'll continue with Track J: software-engineering optimization. I'll use a substantial codebase, profile a concrete bottleneck, and compare a named optimization method with a baseline.

I'm using the public `cachetools` library: its verified source has 1,798 lines. I'll compare greedy and ILP test selection against the full suite, preserving the measured line coverage. I'll also test deliberately mutated copies to check whether a smaller suite misses faults.

The baseline passes all 391 tests, including 24 subtests. The coverage recorder now works from the project folder. I'm measuring test costs next, then checking whether a smaller suite preserves coverage and fault detection.

The full suite exercises 1,005 source lines during test execution. I'll use that as the selection constraint and verify it in fresh runs. Branch coverage and deliberately introduced faults will provide separate checks of what the reduced suites lose.

The ILP selected 74 of 391 tests, and a fresh run preserved all 1,005 required lines. The fault checks already show a limitation: it misses some faults caught by the full suite. I'll report those losses alongside the measured runtime savings.

The timing comparison is complete. The 74-test ILP suite ran 41.5% faster at the median, but lost six branch outcomes and caught 13 of the 16 faults caught by the full suite. One additional mutant timed out for every method; I'm recording it separately from the detected faults.

The audit passed: all 38 upstream files are unchanged, all four coverage matrices match their saved databases, and all 28 timing trials and 80 mutation trials check out. I'm packaging the report, plots, code, raw evidence and viva notes now.

The report and plots are verified, and the earlier nine packages are unchanged. Track J includes reproducibility instructions and two genuine debugging records. After this package, the remaining work is final integration and submission requirements, including the second AI tool comparison.

Thinking

- **Worked**
- **Explored files, ran 8 commands, and searched the web**
- **Explored files, ran 6 commands, and edited 7 files**
- **Explored files, ran 5 commands, and edited 4 files**
- **Edited 4 files and ran 8 commands**
- **Explored files, ran 4 commands, and edited 1 file**
- **Ran 4 commands, explored files, and edited 9 files**
- **Explored files, edited 7 files, and ran 7 commands**
- **Ran 7 commands, edited 1 file, and used Library**

✓ Worked for 32m 37s

Track J is complete: **391 → 74 tests**, with **41.5% lower median runtime**. Measured line coverage was preserved, but the smaller suite missed **3 of 16 faults** caught by the full suite.

`Track J Software Engineering Optimization Project.zip`

Includes code, reports, plots, raw results, tests and viva notes.

All A-J experimental packages are now ready. Next comes the combined report and remaining submission requirements, including the second AI tool comparison.

11/4/2026, 5:56:59 PM · gpt-6.1-sol-wm

continue

11/4/2026, 7:34:40 PM

I’ll combine Tracks A–J into one submission package, check them against the assignment and Chapter 6 material, and build the final report. I’ll also identify anything that still needs your input, such as the second AI tool comparison.

The ten core tracks are present. The chapter checklist also includes topics that need clearer evidence, especially hyperparameter search and communication compression. I’m checking those gaps and adding small experiments where needed. Paper-specific research reproductions will stay clearly marked unless they were actually run.

The added experiments are complete: grid, random and Bayesian search; successive halving and an ASHA trace replay; gradient compression with error feedback; and small scheduling, strategy-search and annealer-embedding examples. I’m now rerunning the existing host tests across A–J before assembling the report.

All **243 existing host tests across A–J passed**. The seven new supplement tests passed too. I’ve prepared a fixed task and common test harness for the second AI tool comparison; that requirement will remain open until there is an actual second-tool response.

Thinking

- **Worked**
- **Explored files, ran 5 commands, searched the web, edited 4 files, and used Library**
- **Ran 9 commands, explored files, searched the web, and edited 5 files**
- **Searched the web, ran 5 commands, and edited 7 files**
- **Ran 6 commands and edited 2 files**

✓ Worked for 24m 36s

11/4/2026, 7:34:47 PM · gpt-6.1-sol-wm

Continue this chat

11/4/2026, 10:58:41 PM

I’m continuing with the combined report and submission package. The code checks are complete; I’m finishing the report layout and final checklist, with the remaining AI comparison and submission steps clearly identified.

The workspace resumed from an earlier snapshot, so I’m restoring the saved track packages and rebuilding the integration files. The ten saved packages are intact; this won’t require you to upload them again.

The restored copies pass all 243 host tests again, and the supplement checks pass. I’ve also prepared a clear inventory of the chapter’s research references so the final checklist won’t confuse a related teaching example with a reproduced paper.

The combined report is now generated: **21 pages** covering A–J, the added chapter experiments and the remaining submission items. I’m checking every page’s layout and building a short captioned demonstration from the actual EV output and saved results.

The page checks passed, and the captioned demo is complete at 100 seconds. All seven available track audit commands also passed in isolated copies, preserving the original evidence. I’m packaging the report, source, raw results and remaining-action checklist now.

The final ZIP passed its checks and a fresh audit after extraction. I’m making the report, combined package and captioned demo available now. The included checklist identifies the remaining second-tool, chat-export, repository and research-scope steps.

Thinking

- **Worked**
- **Ran 2 commands and explored files**
- **Ran 5 commands, edited 7 files, and used Library**
- **Edited 3 files, ran 5 commands, and used Library**
- **Edited 15 files and ran 4 commands**
- **Ran 7 commands and edited 3 files**
- **Edited 5 files and ran 6 commands**
- **Ran 3 commands**

✓ Worked for 26m

The combined A–J package is ready for review:

- 21-page technical report
- Complete project ZIP—code, raw results and checklists
- 100-second captioned demo

243 track tests, 7 supplement tests and all seven audit replays passed.

Before submission, the actual second-tool comparison, full chat/model evidence, GitHub publication and research-paper requirements remain open.

Next: paste the included comparison prompt into Gemini or Claude and send me its response so I can complete the required comparison.

11/4/2026, 10:58:58 PM · gpt-6.1-sol-wm

Yeah, so I will, uh, tell me what should I give? And I will give it to Gemini RCON 4 so that it can give me what. Just give me the what I have to give to R1-4 for the second tool comparison or whatever it is

11/6/2026, 12:19:34 AM

I'll pull out the exact comparison prompt so you can paste it into Gemini.

Thinking

- **Reviewed library documentation content**
- **Cleaning Temporary Library Helper Files**
- **Cleaned temporary library helpers and applied code patches**
- **Prepared assignment comparison materials**
- **Applying metadata to project ZIP file**
- **Applied metadata to project ZIP after reading instructions and README**

✓ Worked for 2m 38s

Paste this into a **new Gemini chat**:

Second AI Tool Comparison Prompt

Write a self-contained Python 3.12 function solve_knapsack(weights, values, capacity), using dynamic programming without third-party libraries.

Return a dictionary with keys value, weight and indices. Indices must be a sorted zero-based list. Each item may be used at most once; total weight must not exceed capacity. Maximize total value, then minimize total weight, then choose lexicographically smallest indices among remaining ties.

Inputs weights and values are lists or tuples of equal length. Weights are positive Python integers; values are nonnegative Python integers. Capacity is a nonnegative Python integer. Booleans, floats, negative capacity, nonpositive weights, negative values and mismatched lengths must raise ValueError. Handle empty inputs, zero capacity and no item fitting. Do not modify inputs. Valid benchmark calls have at most 50 items and capacity <= 10000.

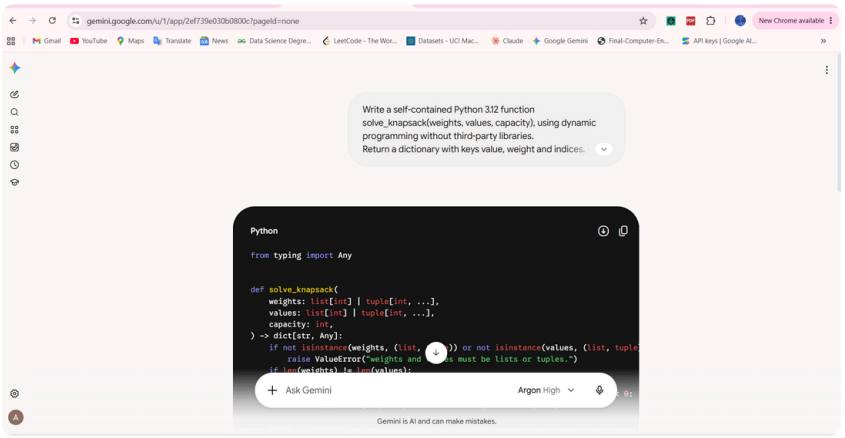
Return only one Python code block containing the implementation and required standard-library imports. No unseen-benchmark claims or external services.

Send me back:

- Its complete first answer, unchanged.
- A screenshot showing the exact model name.
- Roughly how many seconds it took from pressing Send until completion.

I'll test its code against the same checks used for our ChatGPT answer and prepare the comparison.

11/6/2026, 12:19:36 AM · gpt-6.1-sol-wm



answer "```python
from typing import Any

def solve_knapsack(
    weights: list[int] | tuple[int, ...],
    values: list[int] | tuple[int, ...],
    capacity: int,
) -> dict[str, Any]:
    if not isinstance(weights, (list, tuple)) or not isinstance(values, (list, tuple)):
        raise ValueError("weights and values must be lists or tuples.")
    if len(weights) != len(values):
        raise ValueError("weights and values must have the same length.")
    if not isinstance(capacity, int) or isinstance(capacity, bool) or capacity < 0:
        raise ValueError("capacity must be a nonnegative integer.")

    for w in weights:
        if not isinstance(w, int) or isinstance(w, bool) or w <= 0:
            raise ValueError("All weights must be positive integers.")

    for v in values:
        if not isinstance(v, int) or isinstance(v, bool) or v < 0:
            raise ValueError("All values must be nonnegative integers.")

    if not weights or capacity == 0:
        return {"value": 0, "weight": 0, "indices": []}

    dp_val = [-1] * (capacity + 1)
    dp_idx: list[tuple[int, ...]] = [()] * (capacity + 1)
    dp_val[0] = 0

    max_w = 0
    for i, (w, v) in enumerate(zip(weights, values)):
        if w > capacity:
            continue
        max_w = min(capacity, max_w + w)
        for curr_w in range(max_w, w - 1, -1):
            prev_w = curr_w - w
            prev_v = dp_val[prev_w]
            if prev_v != -1:
                cand_v = prev_v + v
                curr_v = dp_val[curr_w]
                if cand_v > curr_v:
                    dp_val[curr_w] = cand_v
                    dp_idx[curr_w] = dp_idx[prev_w] + (i,)
                elif cand_v == curr_v:

```
                cand_idx = dp_idx[prev_w] + (i,)
                if cand_idx < dp_idx[curr_w]:
                    dp_idx[curr_w] = cand_idx

    best_val = 0
    best_weight = 0
    best_indices: tuple[int, ...] = ()

    for w in range(1, max_w + 1):
        if dp_val[w] > best_val:
            best_val = dp_val[w]
            best_weight = w
            best_indices = dp_idx[w]

    return {
        "value": best_val,
        "weight": best_weight,
        "indices": list(best_indices),
    }
```

``" it took it roughly 1 minute to give the answer give or take 2 or 3 seconds

11/6/2026, 12:28:56 AM

I'll test Gemini's original code against the same checks, record its roughly 60-second response time as your estimate, and update the comparison in the report and project package.

Gemini's original answer passed all nine shared tests, including the 100 exhaustive small-case comparisons. No fix was needed. The screenshot shows "Argon High"; I'll use that exact label and keep its estimated response time separate from the measured code runtime.

Both implementations returned the expected answers. In the runtime benchmark, Gemini's function had a lower median on all four cases. I'll report that result separately from its estimated 60-second response time; ChatGPT's original response time wasn't recorded.

The comparison records the screenshot's "Argon High" label, your timing estimate, both unchanged answers and the test results. I'm updating the report and demo to show the completed comparison. Full chat exports, GitHub publication and the instructor's research-reproduction requirement remain open.

The report now has a page with the correctness results, timing table and limits of the comparison. I'm checking the final layouts and package integrity, including confirming that all 2,013 original experiment files are unchanged.

The package checks passed: the 22-page report renders cleanly, the updated demo is 100 seconds, and all original experiment files are unchanged. I'm finalizing the downloadable files with the comparison and test evidence included.

Thinking

- **Worked**
- **Ran 5 commands, edited 6 files, and used Library**
- **Ran 6 commands and explored files**
- **Ran 3 commands and edited 3 files**
- **Edited 1 file, ran 11 commands, and explored files**
- **Ran 7 commands and explored files**
- **Ran 4 commands**

✓ Worked for 16m 14s

Gemini’s original code passed **all 9 shared tests**, including **100 exhaustive small cases**. **No fixes were needed.**

I recorded **“Argon High”** as the displayed label and **approximately 60 ± 3 seconds** as your estimated response time. Both tools’ answers passed; Gemini’s function ran faster on our four benchmark cases. ChatGPT’s generation time wasn’t recorded, so response speed cannot be compared.

Updated files:

- Assignment_8_Integrated_Technical_Report.pdf
- Assignment_8_Integrated_Project.zip
- Assignment_8_Captioned_Demo.mp4

The second-tool comparison is recorded. Full chat exports, GitHub publication and confirmation of the research-reproduction requirements remain.

11/6/2026, 12:29:04 AM · gpt-6.1-sol-wm

do a full check is everything correct and done properly is this ready to submit like can i just download this add it to my drive and share the link with sir and done

11/6/2026, 3:01:17 AM

I'll check the latest report, project ZIP and demo against the assignment requirements, then tell you clearly whether they're ready to submit and whether a Drive link is enough.

The written brief explicitly requires a **public GitHub repository**, so a Drive link alone would not complete the submission. It also requires chat exports or screenshots from each AI tool. I'm checking those evidence gaps alongside the code and report.

I found an omission in the chapter checklist: five quantum research examples from the slides were missing. I'm adding them and marking their experimental status accurately. The full test rerun is also underway; publication and AI-chat evidence remain submission blockers.

The rerun passed all **243 track host tests**, **7 supplement tests**, and **9 tests for each AI candidate**. No original experiment files changed. I'm finishing the corrected report and ZIP; the remaining blockers are the public GitHub repository, AI-platform evidence, and confirmation of whether the research-paper examples also need implementations.

The corrected files include the fresh test logs and a final-review note with the remaining submission steps. The report layout checks passed. I'm checking the rebuilt ZIP before returning the updated downloads.

The rebuilt ZIP passed its final checks. I'm updating the report and project downloads with the corrections and review results.

Thinking

- **Worked**
- **Used Library and ran 4 commands**
- **Ran 6 commands, used Library, and explored files**
- **Ran 6 commands and edited 4 files**
- **Ran 6 commands**
- **Ran 2 commands**
- **Ran 2 commands**

✓ Worked for 9m 26s

Not quite—sharing only a Drive link would not complete the written requirements.

The technical checks passed: **243 track tests, 7 supplement tests and 9 tests per AI candidate**. ZIP integrity and all 2,013 original files checked out. I reviewed the report/demo and corrected five omitted research entries in the checklist.

Before calling it submitted:

1. **Publish the project to a public GitHub repository.** The assignment brief explicitly requires this.
2. **Attach sanitized ChatGPT and Gemini chat exports or screenshots.** Prompt excerpts are included, but platform evidence remains incomplete.
3. **Confirm the research-spotlight scope with sir.** Several paper-specific methods are documented rather than experimentally reproduced; strict coverage could require additional work.

Use these latest files:

- Assignment_8_Integrated_Technical_Report.pdf
- Assignment_8_Integrated_Project.zip
- Assignment_8_Captioned_Demo.mp4

You can put these in Drive now. Before sharing the final submission, include the AI evidence and public GitHub URL, resolve the coverage question, and check that sir can open everything without requesting access.

11/6/2026, 3:01:22 AM · gpt-6.1-sol-wm

1. can i give you the repo name and you push it . 2. how to do what to do guide . 3. tell me and i will ask him in person tomorrow

11/6/2026, 3:14:46 AM

I'll check the GitHub connection, then give you the steps for collecting the AI evidence and the question to ask sir tomorrow.

Thinking

- **Worked**
- **Searched the web and used GitHub**

✓ Worked for 2m 42s

1. Yes—send me the full repository URL, for example:

`https://github.com/akshattikaria/Optimization-Techniques-Assignment-8`

Your GitHub account is connected. Once you send the URL, I'll check access, upload the project folders and verify the published files. If you haven't created it yet, create a **public repository with a README**.

2. For the AI-chat evidence, use screenshots.

Create an `AI_Evidence` folder with two subfolders:

Folder	What to capture
`ChatGPT`	Original project requests and relevant responses, generated code, actual bug/fix discussions and verification results. Include evidence across Tracks A–J and the displayed model label where available.
`Gemini`	The original knapsack prompt, the complete generated function and the visible **"Argon High"** label.

On Windows:

1. Open the **original conversations**.
2. Press **Win + Shift + S** to capture the relevant area.
3. For long responses, take overlapping screenshots so no relevant text is missing.
4. Save readable PNG files with names such as `ChatGPT_Track_A.png` and `Gemini_Answer_01.png`.
5. Exclude private account details, bookmarks and any secrets while keeping the prompt, response and model evidence visible.

Upload those screenshots here. I'll organize them, include them in the project and update the evidence checklist.

3. Tomorrow, ask sir whether the research spotlights require experiments.

Specifically: **Are methods such as AdaFisher, PackStealLB and TensorHyper-VQC compulsory to implement and test, or are they reference material while the main Chapter 6 algorithms require experiments?**

That distinction determines whether the remaining paper-specific experiments are necessary.

11/6/2026, 3:14:51 AM · gpt-6.1-sol-wm

Thinking

- **Reviewing visible context**

  Reviewing visible context

- **Reviewed visible context**
- **Checking reproduction scope**

  Checking reproduction scope
- **Clarifying the requirement**

  Clarifying the requirement
- **Checked reproduction scope and clarified requirements**

✓ Worked for 1m 46s

No—you don’t need to search for sir. That question was my suggested clarification, not a compulsory submission task. I made it sound more mandatory than it is.

I suggested asking because it could save us from implementing research methods that he only intended as references.

We can proceed without asking, using the stricter interpretation:

- Identify the missing research methods.
- Add faithful, appropriately sized implementations and experiments.
- Verify their results and explain their limitations.

That is **additional technical work**, rather than simply adding pages or references. Anything we cannot demonstrate properly must remain clearly identified as incomplete.

A short message could save unnecessary work, but his answer is not required for us to continue improving the assignment.

11/6/2026, 1:14:19 PM · gpt-6.1-sol-wm

good then complete this i will make a github repo and send you its link in next message

11/6/2026, 4:44:52 PM

I'll start closing the missing research-method coverage now. I'll check the papers, add experiments we can verify, and update the report and project package. I'll leave GitHub publication for when you send the repository link.

The papers are identifiable so far, and I found a distinction that matters for the implementation: OFLight uses a server-chosen low-rank projection with error feedback. The existing generic compression example is not enough to claim that method. I'm checking the other papers at the same level before adding code.

The authors' QML-Liver repository is available, including its notebook and dataset. It confirms a two-qubit model, which I'll use for the new experiment. I'll also check the data split and preprocessing so the comparison does not accidentally use test information during training.

The first new experiments have run, and their mathematical checks pass: curvature updates, private knapsack selection, dynamic submodular updates and four quantum-training studies. I also caught a problem in my first AdaFisher port: a single-output model makes its factor normalization degenerate. I'm changing that experiment to a multiclass model and checking it against the paper.

The liver-data experiment has finished. Across the three recorded splits, the quantum model did not show a consistent advantage over the classical controls; I'll retain that result. The subspace experiment also checks that compressed communication preserves the constrained model's training trajectory. Next are search-policy and native sparse-matrix checks.

The SVM+ search experiment and the two-thread sparse-matrix checks have finished. A few papers expose only abstracts or incomplete author repositories, so I'm marking those experiments as demonstrations of the described mechanisms—not exact ports. That distinction will stay visible in the report.

The new experiments pass 20 mathematical checks, and the audit verifies the saved results, data splits and solver certificates. I'm now integrating them into the report and demo, with the six papers that have only mechanism demonstrations clearly identified.

The added studies preserve the mixed results: the liver hybrid loses two of three splits to logistic regression, the tensor-train model performs worse here, and the strategy GA misses the finite model's optimum. Keeping those outcomes makes the comparison more useful and defensible.

The report now has 27 pages, including five pages of new experiments and results. I'm checking the rendered pages and updated video captions before packaging the files. The original EV and A–J results remain unchanged.

The integrated audit passes, including all 2,013 original file hashes and the new research results. The page and video checks are also complete. I'm assembling the download package now; GitHub publication will follow when you send the repository link.

The ZIP passes integrity checks, and its extracted copy passes the audit too. I'm saving the updated project, report and demo. They include the expanded experiments and clearly document the six papers whose exact algorithms still need source access.

Thinking

- **Worked**
- **Used Library, searched the web, and ran 2 commands**
- **Searched the web, ran 4 commands, explored files, and used GitHub**
- **Ran 5 commands, explored files, edited 9 files, searched the web, and used GitHub**
- **Ran 7 commands, edited 7 files, and searched the web**
- **Edited 3 files and ran 3 commands**
- **Searched the web, ran 11 commands, edited 6 files, and explored files**
- **Explored files, ran 8 commands, and edited 4 files**

- **Explored files, ran 16 commands, and edited 4 files**
- **Ran 11 commands and edited 2 files**
- **Ran 28 commands and explored files**
- **Ran 4 commands**

✓ Worked for 59m 39s

I've expanded and checked the assignment: **27-page report, experiments or demonstrations for all 22 research topics, and 20 additional tests passing.** The original A–J work is preserved.

- Updated project ZIP
- Updated report PDF
- Updated demonstration video

Six papers have clearly labelled mechanism demonstrations because their detailed algorithms weren't accessible; exact reproductions aren't claimed. Authentic AI chat exports/screenshots also remain required.

Send the new GitHub repository link next.

11/6/2026, 4:44:53 PM · gpt-6.1-sol-wm

[https://github.com/akshattikaria/OT_LAB_ASSIGNMENT_8]
(https://github.com/akshattikaria/OT_LAB_ASSIGNMENT_8)

11/6/2026, 6:56:41 PM

I'll check the repository, upload the latest project with the report and demo, then verify the published files.

The repository is public and I have write access. I've retrieved the latest saved package and am checking its contents before publishing it.

11/6/2026, 6:56:43 PM · gpt-6.1-sol-wm

i have given you addces to the repo using the githup plugin

11/6/2026, 6:59:13 PM

Yes, the GitHub connection confirms write access to your repository. I’m continuing with the upload.

The package contains all ten tracks, the 27-page report, research extensions, raw results and demo. I’m updating the repository’s README and checklist to record publication while keeping the remaining AI-chat evidence requirement clear.

The integrity audit passed, all 2,013 original track files are unchanged, and 27 fresh supplement tests passed. I’m now uploading the complete source and evidence—about 182 MB—to GitHub.

The upload is still in progress. I’m including the raw data and saved histories along with the code, report and video so the repository contains the full reviewed project.

Automatic approval review blocked publication because it could not confirm permission to make the full package public. The files are prepared; I’m checking the earlier push request before proceeding.

Thinking

- **Worked**
- **Used GitHub and ran 4 commands**
- **Ran 7 commands and used GitHub**
- **Used GitHub and ran 2 commands**
- **Ran 9 commands and used GitHub**
- **Worked**
- **Used GitHub and ran 1 command**
- **Worked**
- **Used GitHub and Personal Context and ran 1 command**

✓ Worked for 3m 58s

Write access is confirmed, and the package is prepared and verified.

Please reply **“Publish the full assignment publicly”** to approve uploading its code, results, report, demo and saved histories to your repository.

Automatic approval review blocked publication because it could not confirm authorization to make the complete package public.

11/6/2026, 6:59:14 PM · gpt-6.1-sol-wm

Publish the full assignment publicly

11/6/2026, 7:07:41 PM

I'll publish the complete assignment to your repository, then verify that the code, results, report and demo are accessible.

The files are transferring in batches. Once the transfer finishes, I'll update `main` and check the published files against the prepared package.

The report and initial source batches have transferred. I'm continuing with the remaining code and the larger result files.

The main source-code batches are transferred. I'm now transferring the raw experiment results, databases and history bundles, which make up most of the package.

The transfer is still in progress, including the saved binary evidence files. After it finishes, I'll verify every published file against the prepared package.

I'm continuing the transfer and checking each file's Git hash as it arrives. The larger saved results and binary files are taking most of the time.

The transfer hit an issue with longer file reads. I'm correcting the reader and will resume from the files already verified; the prepared package is unchanged.

The reader correction is working, and the transfer has resumed. Completed files are matching their expected hashes.

The evidence-file uploads are progressing slowly, but GitHub is accepting them and the completed transfers match their expected hashes. I'm continuing from the saved progress.

Saved test-run records and more datasets have transferred successfully. I'm continuing with the remaining plots, result files and histories.

The transfer is still running. Verified uploads are saved, and I'm working through the remaining evidence files.

82 of the remaining 213 unique files have transferred. I'm continuing with the rest before publishing and checking the complete repository.

137 of 213 remaining unique files are transferred, including more result tables and plots. The larger result files, databases and history bundles are still in progress.

11/6/2026, 7:07:46 PM · gpt-6.1-sol-wm